INNOMATICS RESEARCH LABS

Internship Project Report

RAG-BASED CUSTOMER SUPPORT ASSISTANT (WITH LANGGRAPH & HITL)

Final Review Project

Name	Amritha Suvarna
Usn	4MW22AD005
University	Visvesvaraya Technological University (VTU)
College	Shri Madhwa Vadiraja Institute of Technology and Management, Bantakal, Udupi

List of Acronyms and Abbreviations

RAG	Retrieval-Augmented Generation
LLM	Large Language Model
HITL	Human-in-the-Loop
CLI	Command Line Interface
PDF	Portable Document Format
HLD	High-Level Design
LLD	Low-Level Design
API	Application Programming Interface
DB	Database
UI	User Interface
NLP	Natural Language Processing
AI	Artificial Intelligence
ML	Machine Learning
k (Top-k)	Number of top relevant results retrieved
ctx	Context
conf	Confidence Score
ans	Answer
q	Query

Design & Build a RAG-Based Customer Support Assistant (with LangGraph & HITL)

1. INTRODUCTION

1.1 Context and Problem Overview

In today's digital ecosystem, organizations generate a vast amount of documentation such as FAQs, manuals, and support guides. However, extracting meaningful information from these documents quickly remains a challenge for users. Traditional keyword-based search systems often fail to provide precise answers, leading to poor user experience.

To address this limitation, modern AI systems use Retrieval-Augmented Generation (RAG), which combines retrieval mechanisms with generative models to produce accurate and context-aware responses.

1.2 What is Retrieval-Augmented Generation (RAG)?

RAG is a hybrid architecture that integrates two key components:

- Retriever: Fetches relevant information from a knowledge base
- Generator (LLM): Produces human-like responses based on retrieved context

Unlike standalone language models, RAG systems:

- Do not rely solely on pre-trained knowledge
- Dynamically retrieve relevant information
- Provide more factual and grounded responses

1.3 Problem Statement

Users often struggle to:

- Navigate large PDF documents

- Extract precise answers quickly
- Trust AI responses due to hallucinations

This project aims to solve these issues by building an intelligent assistant that:

- Understands user queries
- Retrieves relevant document content
- Generates accurate answers
- Escalates to humans when necessary

1.4 Objectives

The main objectives of this project are:

- To design a RAG-based customer support system
- To implement document ingestion and retrieval
- To integrate LLM-based answer generation
- To build a graph-based workflow using LangGraph
- To implement Human-in-the-Loop (HITL) escalation

2. HIGH-LEVEL DESIGN (HLD)

2.1 System Overview

The system is designed as an intelligent pipeline that transforms static documents into an interactive question-answering system.

Key Features:

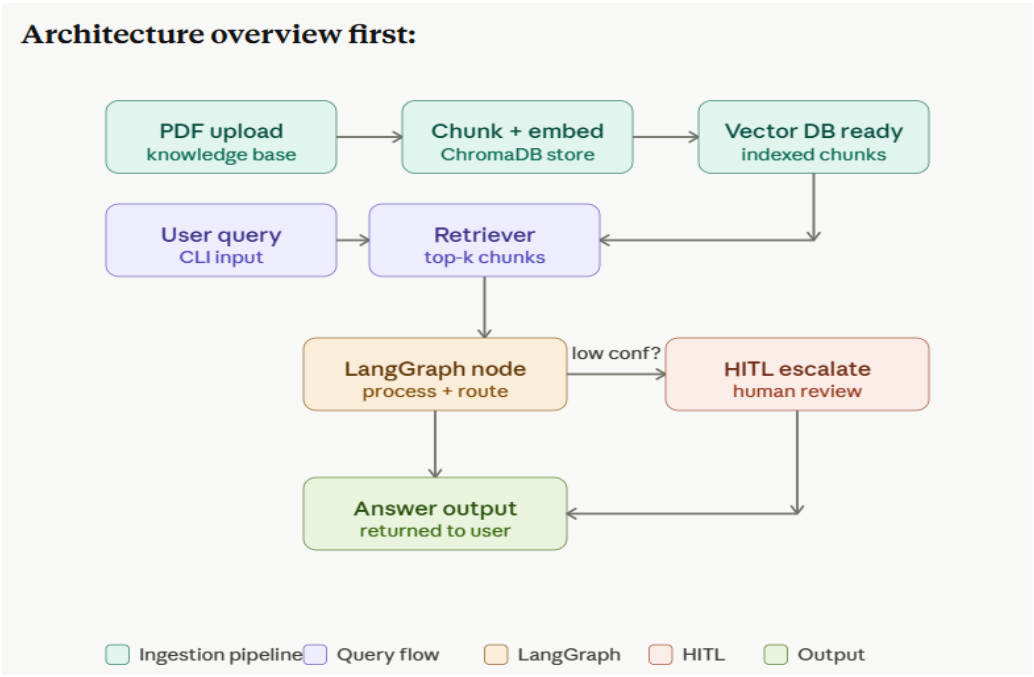
- Accepts PDF as knowledge base
- Processes and stores semantic embeddings
- Retrieves relevant context for queries
- Generates responses using LLM
- Escalates uncertain queries to human support

2.2 Architectural Design

The architecture follows a modular and layered approach:

1. User Interface Layer
   - CLI-based interaction
   - Accepts user queries
2. Processing Layer
   - Handles query processing
   - Retrieves relevant document chunks
3. Intelligence Layer
   - Uses LLM to generate answers
4. Control Layer
   - LangGraph workflow orchestration
   - Decision-making (routing logic)
5. Human Support Layer
   - Handles escalations

2.3 Architecture Flow Explanation



The complete system workflow can be understood as follows:

1. User enters a query
2. Query is passed into LangGraph workflow
3. Retriever fetches top relevant chunks from vector DB
4. Context is constructed
5. Confidence score is calculated
6. Routing decision is made:
   - If confident, LLM generates answer
   - If not, escalate to human
7. Final response is returned

2.4 Component Description

2.4.1 Document Loader

The document loader is responsible for ingesting PDF files and converting them into structured text format.

- Tool used: PyPDFLoader
- Output: List of document objects with metadata
- Importance: Enables downstream processing

2.4.2 Chunking Strategy

Large documents cannot be processed efficiently as a whole. Hence, they are split into smaller chunks.

- Method: RecursiveCharacterTextSplitter
- Chunk size: 500 characters
- Overlap: 50 characters

Overlap is important because, it ensures continuity of context between chunks, preventing loss of meaning.

2.4.3 Embedding Model

Embeddings convert textual data into numerical vectors.

- Model: all-MiniLM-L6-v2
- Type: Sentence Transformer

Advantage:

- Lightweight
- Fast
- Good semantic understanding

2.4.4 Vector Database (ChromaDB)

The vector database stores embeddings and enables similarity search.

- Stores high-dimensional vectors
- Supports fast nearest-neighbor search
- Returns semantically similar chunks

2.4.5 Retriever

The retriever fetches the most relevant chunks for a given query.

- Strategy: Top-k similarity search
- k = 3
- Output: Most relevant document chunks

2.4.6 Language Model (LLM)

The LLM generates answers based on retrieved context.

- Model used: LLaMA 3.1 via Groq
- Key Features:
  - Fast inference
  - Context-aware responses
  - Low latency

2.4.7 LangGraph Workflow Engine

LangGraph controls the execution flow using nodes and edges.

- Enables modular design
- Supports conditional routing
- Maintains system state

2.4.8 Routing Layer

The routing mechanism determines whether:

- The system can answer confidently
- Or needs human intervention

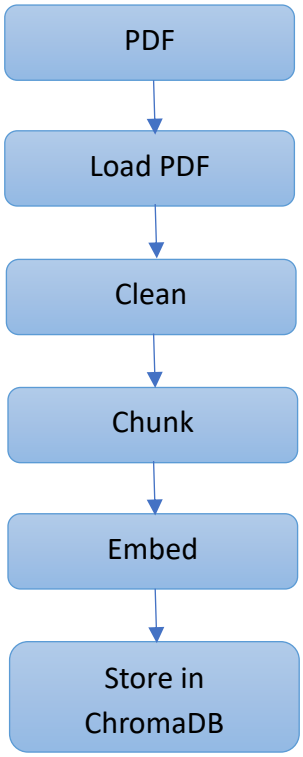
2.4.9 Human-in-the-Loop (HITL)

HITL ensures system reliability by:

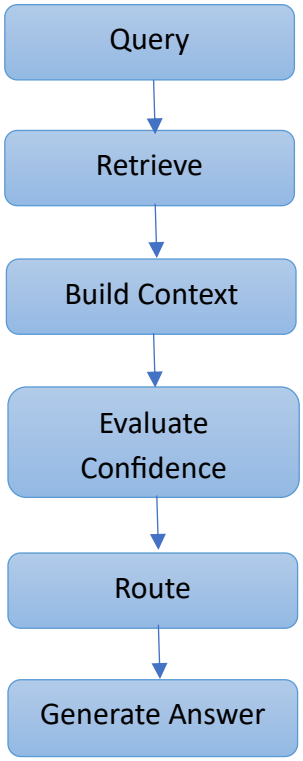
- Handling uncertain queries
- Allowing human validation
- Providing fallback responses

2.5 Data Flow

Document Processing Flow:



Query Processing Flow:



2.6 Technology Stack Justification

Component	Technology	Justification
Loader	PyPDFLoader	Efficient PDF parsing
Chunking	LangChain Splitter	Flexible splitting
Embeddings	MiniLM	Fast and accurate
Vector DB	ChromaDB	Lightweight and scalable
LLM	Groq LLaMA	Low latency
Workflow	LangGraph	Graph-based control

2.7 Scalability Considerations

- Large Documents: Chunking ensures scalability
- High Queries: Vector DB supports fast retrieval
- Latency Optimization:
  - Smaller models
  - Reduced chunk size
- Future Scaling:
  - Distributed databases
  - API deployment
  - Cloud integration

3. LOW-LEVEL DESIGN (LLD)

3.1 Module-Level Breakdown

The system is divided into independent modules:

Document Module

- Handles loading and preprocessing

Chunking Module

- Splits text efficiently

Embedding Module

- Generates vector representations

Storage Module

- Stores embeddings in ChromaDB

Retrieval Module

- Fetches relevant chunks

Query Module

- Builds prompts and interacts with LLM

Graph Module

- Controls workflow execution

HITL Module

- Manages human escalation

3.2 Data Structures

Document Object

Stores raw text and metadata.

Chunk Structure

Contains:

- Text content
- Vector embedding

State Object (Core)

```
{
  "q": "user query",
  "ctx": "retrieved context",
  "ans": "final answer",
  "route": "answer/escalate",
  "conf": 0.0
}
```

3.3 Workflow Design

Nodes:

- process_node: retrieval and decision
- answer_node: generate answer
- hitl_node: escalation

State Transitions:

- Input → Context → Decision → Output

3.4 Conditional Routing Logic

AI Answer Conditions:

- Context length sufficient
- Confidence ≥ threshold (0.3)

Escalation Conditions:

- Low context
- Ambiguous query
- Irrelevant input

3.5 HITL Workflow

1. Low confidence detected
2. Query escalated
3. Human provides response
4. System returns final answer

3.6 Interface Design

Input:

- User query via CLI (Command Line Interface)

Output:

- Generated answer
- Confidence score

3.7 Error Handling Strategy

- No relevant chunks → escalate
- Empty input → ignore
- LLM failure → retry or fallback
- Invalid data → default response

4. DESIGN DECISIONS

Chunk Size Selection

Chosen to balance:

- Context richness
- Processing speed

Embedding Model Choice

MiniLM selected for:

- Speed
- Efficiency
- Good semantic capture

Retrieval Strategy

Top-k retrieval ensures:

- Relevant context
- Reduced noise

Prompt Design

Structured prompt ensures:

- Clarity
- Better LLM output

5. WORKFLOW EXPLANATION

LangGraph enables structured execution:

- Each node performs a specific task
- State flows between nodes
- Routing decisions are dynamic

This improves modularity and debugging.

6. HITL ANALYSIS

Benefits:

- Improves reliability
- Prevents incorrect answers
- Adds human oversight

Limitations:

- Slower responses
- Requires manual effort

7. CHALLENGES & TRADE-OFFS

Challenge	Trade-off
Accuracy vs Speed	Larger context slows system
Chunk size	Bigger chunks, better context but slower
Cost	Better models, higher cost

8. TESTING STRATEGY

Approach:

- Functional testing
- Query-based validation

Sample Queries:

- Valid queries
- Irrelevant queries
- Edge cases

9. FUTURE ENHANCEMENTS

- Multi-document support
- Memory-based chat
- Web UI integration
- API deployment
- Feedback learning

10. CONCLUSION

This project successfully demonstrates a real-world implementation of a RAG-based customer support system. It integrates retrieval, generation, and decision-making into a unified workflow. The use of LangGraph enhances modularity, while HITL ensures reliability. The system is scalable, efficient, and practical for real-world deployment scenarios.